

```

# Google Colab emotion classifier training
# Google Colab was used because training the DistilBERT model locally was taking too much time
# The trained model is saved and downloaded (under the name emotion_model in Gublu/) so it can be used later in the Gublu ba

# Step 1 = Installing the required libraries
# These libraries are needed for transformers, datasets, model training, metrics and data handling
!pip install transformers datasets torch scikit-learn pandas accelerate -q

# Step 2 = Importing the required libraries
import pandas as pd # Importing pandas to read and handle the CSV dataset files
import torch # Importing torch because the model training uses PyTorch tensors
from torch.utils.data import Dataset # Importing Dataset to create a custom dataset class for training
from transformers import ( # Importing the Hugging Face tools needed for tokenising, loading and training the model
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments
)
from sklearn.metrics import accuracy_score, precision_recall_fscore_support # Importing metrics to evaluate the model perfo

# Step 3 = Loading the CSV dataset files
# Before running this part, train_emotion.csv, val_emotion.csv and test_emotion.csv need to be uploaded to Colab
train_df = pd.read_csv("train_emotion.csv") # Loading the training dataset
val_df = pd.read_csv("val_emotion.csv") # Loading the validation dataset
test_df = pd.read_csv("test_emotion.csv") # Loading the testing dataset

print("Datasets loaded successfully!") # Printing this to confirm the datasets loaded properly

print("Train labels:", sorted(train_df["label"].unique())) # Checking which labels exist in the training dataset

# Step 4 = Defining the emotion labels
# These are the emotion categories used by the Gublu emotion model
label_list = ["sadness", "joy", "anger", "fear", "surprise"] # Setting the emotion labels used for training

label2id = {label: i for i, label in enumerate(label_list)} # Converting each label name into a number for the model
id2label = {i: label for label, i in label2id.items()} # Converting each number back into the label name

# Step 5 = Creating the custom dataset class
# This class changes the dataframe into a format that the Hugging Face Trainer can use

class EmotionDataset(Dataset): # Creating a custom dataset class for emotion training

    def __init__(self, dataframe, tokenizer): # Setting up the dataset using the dataframe and tokenizer
        self.texts = dataframe["text"].tolist() # Storing all text examples from the dataset
        self.labels = dataframe["label"].tolist() # Storing all emotion labels from the dataset
        self.tokenizer = tokenizer # Storing the tokenizer so it can process each text example

    def __len__(self): # This function returns how many examples are in the dataset
        return len(self.texts) # Returning the number of text examples

    def __getitem__(self, idx): # This function gets one dataset item at a time
        text = self.texts[idx] # Getting the text at the current index
        label = self.labels[idx] # Getting the label at the current index

        encoding = self.tokenizer( # Tokenising the text so the DistilBERT model can read it
            text,
            padding="max_length", # Padding all texts to the same length
            truncation=True, # Cutting text if it is longer than the maximum length
            max_length=128, # Setting the maximum token length for each text
            return_tensors="pt" # Returning PyTorch tensors
        )

        return { # Returning the processed text and its label
            "input_ids": encoding["input_ids"].squeeze(), # Returning the token IDs without the extra dimension
            "attention_mask": encoding["attention_mask"].squeeze(), # Returning the attention mask without the extra dimens
            "labels": torch.tensor(label) # Returning the label as a tensor
        }

# Step 6 = Loading the tokenizer
# The tokenizer changes normal text into tokens that DistilBERT understands
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased") # Loading the DistilBERT tokenizer

# Step 7 = Creating the train, validation and test datasets
# These datasets are created using the custom EmotionDataset class
train_dataset = EmotionDataset(train_df, tokenizer) # Creating the training dataset

```

```

train_dataset = EmotionDataset(train_df, tokenizer) # Creating the training dataset
val_dataset = EmotionDataset(val_df, tokenizer) # Creating the validation dataset
test_dataset = EmotionDataset(test_df, tokenizer) # Creating the testing dataset

# Step 8 = Loading the DistilBERT model
# This loads the pretrained model and changes the final layer for the five emotion labels

model = AutoModelForSequenceClassification.from_pretrained( # Loading DistilBERT for emotion classification
    "distilbert-base-uncased",
    num_labels=len(label_list), # Setting the number of emotion classes
    id2label=id2label, # Giving the model the number-to-label mapping
    label2id=label2id # Giving the model the label-to-number mapping
)

# Step 9 = Creating the metrics function
# This function calculates the model's accuracy, precision, recall and F1 score during evaluation

def compute_metrics(pred): # Creating the function used by the Trainer to calculate metrics
    logits, labels = pred # Getting the raw model outputs and the real labels
    predictions = logits.argmax(axis=1) # Choosing the label with the highest score as the model prediction

    precision, recall, f1, _ = precision_recall_fscore_support( # Calculating precision, recall and F1
        labels, predictions, average="weighted" # Using weighted average because the emotion labels may not be perfectly balanced
    )

    acc = accuracy_score(labels, predictions) # Calculating the accuracy score

    return { # Returning all metrics in a dictionary
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }

# Step 10 = Setting the training configuration
# These settings control how the model trains

training_args = TrainingArguments(
    output_dir="./results", # Saving training outputs in the results folder
    learning_rate=2e-5, # Setting how quickly the model updates while learning
    per_device_train_batch_size=16, # Setting how many training examples are processed at once
    per_device_eval_batch_size=16, # Setting how many validation examples are processed at once
    num_train_epochs=3, # Training the model over the dataset 3 times
    eval_strategy="epoch", # Running validation after each epoch
    save_strategy="epoch" # Saving the model after each epoch
)

# Step 11 = Creating the Trainer
# The Trainer handles the training loop, evaluation and saving process

trainer = Trainer(
    model=model, # Giving the Trainer the DistilBERT model
    args=training_args, # Giving the Trainer the training settings
    train_dataset=train_dataset, # Giving the Trainer the training dataset
    eval_dataset=val_dataset, # Giving the Trainer the validation dataset
    processing_class=tokenizer, # Giving the Trainer the tokenizer
    compute_metrics=compute_metrics # Giving the Trainer the metrics function
)

# Step 12 = Training the model
# This starts the fine-tuning process

print("Training started...") # Printing this before training begins
trainer.train() # Training the emotion classifier

# Step 13 = Evaluating the trained model
# This checks how well the model performs on validation and test data

print("\nValidation Results:") # Printing the validation results heading
val_results = trainer.evaluate() # Evaluating the model on the validation dataset
print(val_results) # Printing the validation metrics

print("\nTest Results:") # Printing the test results heading
test_results = trainer.evaluate(test_dataset) # Evaluating the model on the testing dataset
print(test_results) # Printing the test metrics

```

```
# Step 14 = Saving the trained model
# This saves the model and tokenizer into an emotion_model folder

trainer.save_model("emotion_model") # Saving the trained model
tokenizer.save_pretrained("emotion_model") # Saving the tokenizer with the model

print("Model saved!") # Printing this to confirm the model was saved

# Step 15 = Downloading the trained model
# This zips the model folder so it can be downloaded from Google Colab

import shutil # Importing shutil to create a zip file
from google.colab import files # Importing files to download the zip file from Colab

shutil.make_archive("emotion_model", 'zip', "emotion_model") # Zipping the saved model folder
files.download("emotion_model.zip") # Downloading the zipped model file

# Step 16 = Testing a sample prediction
# This checks if the saved model can classify a new emotional sentence

from transformers import pipeline # Importing pipeline to make quick predictions with the saved model

classifier = pipeline(
    "text-classification", # Setting the task as text classification
    model="emotion_model", # Loading the saved emotion model
    tokenizer="emotion_model", # Loading the saved tokenizer
    top_k=None # Returning all emotion scores instead of only the highest one
)

def predict_emotions(text): # Creating a function to predict emotion scores from a text message
    results = classifier(text)[0] # Running the text through the classifier and getting the results

    scores = {item["label"].lower(): item["score"] for item in results} # Changing the results into a simple label score dict

    final_scores = { # Creating the final emotion score format used by Gublu
        "sadness": scores.get("sadness", 0),
        "joy": scores.get("joy", 0),
        "anger": scores.get("anger", 0),
        "fear": scores.get("fear", 0),
        "surprise": scores.get("surprise", 0),
        "neutral": 0.0 # Adding neutral manually because the trained dataset did not include a neutral class
    }

    total = sum(final_scores.values()) # Adding all scores together so they can be normalised

    if total > 0: # Checking that the total is not zero before dividing
        final_scores = {k: v / total for k, v in final_scores.items()} # Normalising the scores so they add up to 1

    return final_scores # Returning the final emotion scores

# Testing the trained emotion classifier with one sample sentence
print("\nSample Prediction:") # Printing the sample prediction heading
print(predict_emotions("I feel really anxious and overwhelmed")) # Printing the emotion scores for the sample sentence
```

```
Datasets loaded successfully!
Train labels: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4)]
Loading weights: 100% 100/100 [00:00<00:00, 155.08it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
pre_classifier.weight | MISSING |
classifier.bias | MISSING |
pre_classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task
Training started...
[3000/3000 09:45, Epoch 3/3]

Epoch Training Loss Validation Loss Accuracy F1 Precision Recall
-----
1 0.190559 0.157338 0.944000 0.944242 0.945592 0.944000
2 0.107521 0.125547 0.958500 0.958711 0.959373 0.958500
3 0.065352 0.125913 0.957000 0.956934 0.957310 0.957000

Writing model shards: 100% 1/1 [00:06<00:00, 6.67s/it]
Writing model shards: 100% 1/1 [00:05<00:00, 5.65s/it]
Writing model shards: 100% 1/1 [00:05<00:00, 5.32s/it]

Validation Results:
[125/125 00:15]
{'eval_loss': 0.12591321766376495, 'eval_accuracy': 0.957, 'eval_f1': 0.9569335602371323, 'eval_precision': 0.95730989967782

Test Results:
{'eval_loss': 0.12856216728687286, 'eval_accuracy': 0.9535, 'eval_f1': 0.9531869374405533, 'eval_precision': 0.9532224167932

Writing model shards: 100% 1/1 [00:05<00:00, 5.62s/it]
Model saved!
Loading weights: 100% 104/104 [00:00<00:00, 479.34it/s, Materializing param=pre_classifier.weight]
```